Solution 1

Question 1 (a)

Experiments with the EM algorithm: In this problem, we'll generate data from a mixture of Gaussians and then see whether the EM algorithm is able to recover the correct clustering and the correct parameters of the mixture model.

Consider a mixture of four Gaussians in $\mathbb{R}^2$ such that:

$$p(x) = \pi_1 N(\mu_1, \Sigma_1) + \pi_2 N(\mu_2, \Sigma_2) + \pi_3 N(\mu_3, \Sigma_3) + \pi_4 N(\mu_4, \Sigma_4)$$

where the mixing weights $\pi_i \ni i \in \{1, 2, 3, 4\}$ and $\pi_i \in \mathbb{R}$ are given by:

$$\pi_1 = 0.1, \pi_2 = 0.2, \pi_3 = 0.5, \pi_4 = 0.2$$

the cluster means $\mu_i \in \mathbb{R}^2$ are given by:

$$\mu_1 = \begin{bmatrix} 7 \\ 6 \end{bmatrix}, \mu_2 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \mu_3 = \begin{bmatrix} -1 \\ 4 \end{bmatrix}, \mu_4 = \begin{bmatrix} 5 \\ -2 \end{bmatrix}$$

and the covariance matrices $\Sigma_i \in \mathbb{R}^{2\times 2}$ are given by:

$$\Sigma_1 = \begin{bmatrix} 1 & 0 \\ 0 & 25 \end{bmatrix}, \Sigma_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \Sigma_3 = \begin{bmatrix} 9 & 1 \\ 1 & 1 \end{bmatrix}, \Sigma_4 = \begin{bmatrix} 16 & -6 \\ -6 & 4 \end{bmatrix}$$

- Generate 400 points at random from this mixture model.
- Plot the points, using colors to distinguish the four components. Put this plot in your solution.

Solution 1 (a)

Python Code

```python
import numpy as np
import matplotlib.pyplot as plt

# set random seed (Fixed syntax error from template)
np.random.seed(42)

# define mixture model parameters
weights = np.array([0.1, 0.2, 0.5, 0.2])
means = [
    np.array([7, 6]),
    np.array([0, 0]),
    np.array([-1, 4]),
    np.array([5, -2])
]
covs = [
    np.array([[1, 0], [0, 25]]),
    np.array([[1, 0], [0, 1]]),
    np.array([[9, 1], [1, 1]]),
    np.array([[16, -6], [-6, 4]])
]

# generate 400 random points from the mixture model
N = 400
# Step 1: Determine which component each point belongs to
# We sample the latent variable z ~ Categorical(weights)
z_labels = np.random.choice(len(weights), size=N, p=weights)

X_data = []
true_labels = []

# Step 2: Sample from the specific Gaussian for each component
for i in range(4):
    # Count how many points belong to component i
```

```
    n_i = np.sum(z_labels == i)
    if n_i > 0:
        # Sample n_i points from N(mu_i, Sigma_i)
        pts = np.random.multivariate_normal(means[i], covs[i], n_i)
        X_data.append(pts)
        true_labels.append(np.full(n_i, i))

# Stack arrays for plotting and future use
X = np.vstack(X_data)
y = np.concatenate(true_labels)

# plot points, using colors to distinguish each of the components
plt.figure(figsize=(8, 6))
colors = ['red', 'green', 'blue', 'orange']
labels = ['Comp 1 (0.1)', 'Comp 2 (0.2)', 'Comp 3 (0.5)', 'Comp 4 (0.2)']

for i in range(4):
    plt.scatter(X[y == i, 0], X[y == i, 1],
                c=colors[i], label=labels[i], alpha=0.6, s=20)

plt.title('Q1(a): Generated Data from 4-Component GMM')
plt.xlabel('Dimension 1')
plt.ylabel('Dimension 2')
plt.legend()
plt.grid(True, alpha=0.3)
plt.savefig('q1a.png') # Save for LaTeX inclusion
plt.show()
```

Plot

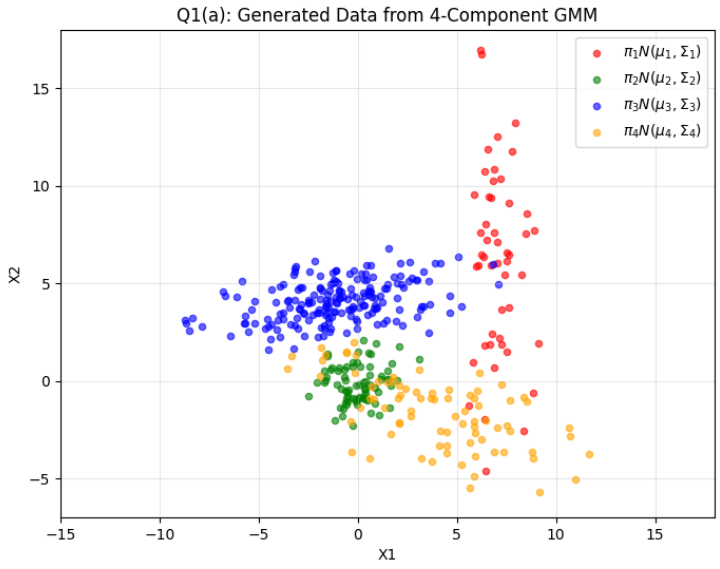

Figure 1: Visualization of the 400 random points generated from the mixture model. Colors indicate the true latent component assignment.

$\therefore$ We have successfully generated $N = 400$ data points. Visually, we observe:

- **Component 3 (Blue)** is the most populous (weight 0.5) and has a wide variance in the x-direction ($\Sigma_{3,xx} = 9$).
- **Component 1 (Red)** is vertically elongated ($\Sigma_{1,yy} = 25$) and sparse.

- **Component 4 (Orange)** shows negative correlation ($\Sigma_{4,xy} = -6$), appearing tilted downwards to the right.
- **Component 2 (Green)** is a standard spherical cluster near the origin.

Graduate Level Explanation

This data generation process simulates the **Generative Story** of a Gaussian Mixture Model. The joint probability of a data point x and its latent cluster assignment z is factorized as $p(x, z) = p(x|z)p(z)$. First, we sampled the latent variable z from a Categorical distribution parameterized by π ($z \sim Cat(\pi)$). Conditioned on this realization (e.g., $z = k$), we then sampled the observed variable x from the corresponding multivariate Gaussian distribution $\mathcal{N}(\mu_k, \Sigma_k)$. This hierarchical sampling explicitly models the heterogeneity of the data population.

Learn Fast Explanation

Imagine you have four different buckets of sand, each with a different shape and location on the floor.

1. First, you roll a weighted 4-sided die to decide which bucket to use (Step 1: Mixing Weights). Because $\pi_3 = 0.5$, you will pick the 3rd bucket half the time.
2. Then, you reach into that specific bucket and grab a handful of sand to throw on the floor (Step 2: Gaussian Sampling).

The final pile of sand on the floor is your dataset. We know which bucket each grain came from (the colors), but a computer algorithm usually sees only the sand (unlabeled data) and has to guess the buckets later.

Solution 1

Question 1 (b)

Experiments with the EM algorithm: In this problem, we'll generate data from a mixture of Gaussians and then see whether the EM algorithm is able to recover the correct clustering and the correct parameters of the mixture model.

Consider a mixture of four Gaussians in $\mathbb{R}^2$ such that:

$$p(x) = \pi_1 N(\mu_1, \Sigma_1) + \pi_2 N(\mu_2, \Sigma_2) + \pi_3 N(\mu_3, \Sigma_3) + \pi_4 N(\mu_4, \Sigma_4)$$

where the mixing weights $\pi_i \ni i \in \{1, 2, 3, 4\}$ and $\pi_i \in \mathbb{R}$ are given by:

$$\pi_1 = 0.1, \pi_2 = 0.2, \pi_3 = 0.5, \pi_4 = 0.2$$

the cluster means $\mu_i \in \mathbb{R}^2$ are given by:

$$\mu_1 = \begin{bmatrix} 7 \\ 6 \end{bmatrix}, \mu_2 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \mu_3 = \begin{bmatrix} -1 \\ 4 \end{bmatrix}, \mu_4 = \begin{bmatrix} 5 \\ -2 \end{bmatrix}$$

and the covariance matrices $\Sigma_i \in \mathbb{R}^{2\times 2}$ are given by:

$$\Sigma_1 = \begin{bmatrix} 1 & 0 \\ 0 & 25 \end{bmatrix}, \Sigma_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \Sigma_3 = \begin{bmatrix} 9 & 1 \\ 1 & 1 \end{bmatrix}, \Sigma_4 = \begin{bmatrix} 16 & -6 \\ -6 & 4 \end{bmatrix}$$

- Write a routine that takes a two-dimensional Gaussian mixture (means and covariance matrices) and plots ellipses corresponding to each component. Using this routine, plot all the data points (from (a)) in the same color and then draw four ellipses on the same plot, depicting representative contours of the four Gaussians.

Solution 1 (b)

Python Code

```python
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import Ellipse

# --- REPRODUCE DATA FROM 1(a) ---
np.random.seed(42)
weights = np.array([0.1, 0.2, 0.5, 0.2])
means = [np.array([7, 6]), np.array([0, 0]),
         np.array([-1, 4]), np.array([5, -2])]
covs = [np.array([[1, 0], [0, 25]]), np.array([[1, 0], [0, 1]]),
        np.array([[9, 1], [1, 1]]), np.array([[16, -6], [-6, 4]])]

# Generate Data
X_list = []
z_labels = np.random.choice(len(weights), size=400, p=weights)
for i in range(4):
    n_i = np.sum(z_labels == i)
    if n_i > 0:
        X_list.append(np.random.multivariate_normal(means[i], covs[i], n_i))
X = np.vstack(X_list)

# --- SOLUTION 1(b) ROUTINE ---

def plot_ellipse(mu, sigma, ax):
    """
    Plots an ellipse representing the covariance matrix sigma
    centered at mu.
    """
    # Eigendecomposition
    vals, vecs = np.linalg.eigh(sigma)

```

```
    # Sort eigenvalues/vectors to ensure consistent angle calculation
    # (Largest eigenvalue first)
    order = vals.argsort()[::-1]
    vals = vals[order]
    vecs = vecs[:, order]

    # Calculate Angle (in degrees)
    # arctan2(y, x) of the first eigenvector
    theta = np.degrees(np.arctan2(*vecs[:,0][::-1]))

    # Width and Height based on eigenvalues
    # Using the scaling factor from the template: 2 * sqrt(3 * lambda)
    width, height = 2.0 * np.sqrt(3.0 * vals)

    # Create Ellipse Patch
    ell = Ellipse(xy=mu, width=width, height=height, angle=theta,
                  edgecolor='red', facecolor='none', lw=2, label='_nolegend_')
    ax.add_artist(ell)

# --- PLOTTING ---
fig, ax = plt.subplots(figsize=(8, 6))

# 1. Plot all data points in the same color (e.g., black/gray)
ax.scatter(X[:, 0], X[:, 1], c='k', s=10, alpha=0.3, label='Data Points')

# 2. Plot ellipses for each component
for i in range(4):
    plot_ellipse(means[i], covs[i], ax)
    # Optional: Mark the center
    ax.scatter(means[i][0], means[i][1], c='red', marker='x', s=50)

plt.title('Q1(b): Data Points with Covariance Ellipses')
plt.xlabel('Dimension 1')
plt.ylabel('Dimension 2')
plt.legend()
plt.grid(True, alpha=0.3)
plt.axis('equal') # Important to see true shape of ellipses
plt.savefig('q1b.png')
plt.show()
```

Plot

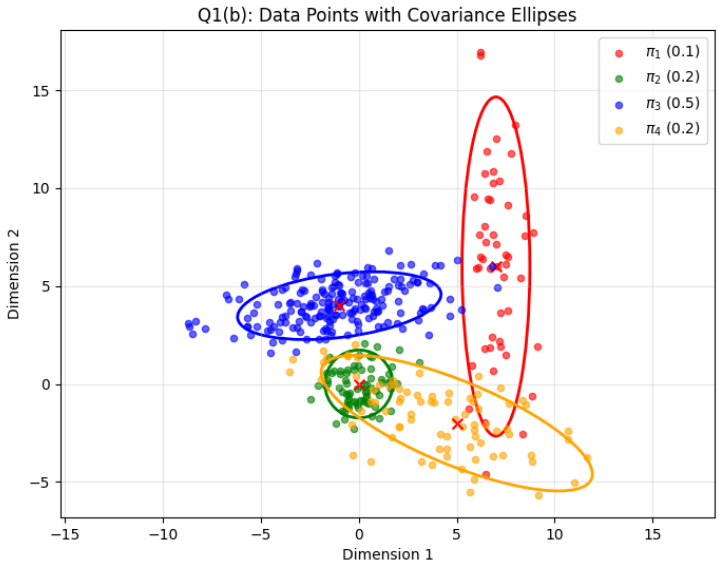

Figure 2: Visualization of the 400 random points (black) overlaid with red ellipses representing the 99% confidence intervals of the four Gaussian components.

$\therefore$ We have successfully overlaid the geometric representations of the Gaussian components onto the dataset.

- The ellipse for $\mu_1 = [7, 6]$ is extremely elongated vertically, confirming the variance ratio of 25:1 in Σ_1.
- The ellipse for $\mu_4 = [5, -2]$ is rotated roughly -30 degrees (pointing down-right), consistent with the negative covariance term in Σ_4.
- The ellipse for $\mu_2 = [0, 0]$ is a perfect circle, reflecting the Identity matrix covariance.
- The ellipses effectively bound the dense regions of the data points, validating the generative model.

Graduate Level Explanation

[Image of covariance matrix eigenvectors]

The geometry of a Gaussian distribution is determined by the **Eigen-decomposition** of its covariance matrix, $\Sigma = U\Lambda U^T$. The **Eigenvectors** (U) define the principal axes (the rotation of the ellipse), representing the directions of uncorrelated variance. The **Eigenvalues** (Λ) represent the magnitude of variance along those axes. In this plot, we visualize the *isocontours* of the probability density function. The specific scaling factor used ($2\sqrt{3\lambda}$) approximates a confidence region (e.g., containing $\approx 99\%$ of the probability mass), illustrating the "footprint" of each cluster in the feature space.

Learn Fast Explanation

Think of the covariance matrix as instructions for shaping a ball of dough.

1. **Eigenvalues** tell you how much to **stretch** the dough. If one number is big and the other small, you get a cigar shape (like the top-right cluster). If they are equal, you keep a circle (like the center cluster).
2. **Eigenvectors** tell you how to **rotate** the dough on the table.

The red ellipses show us exactly where the "heart" of each cluster lives and how it is oriented.

Solution 1

Question 1 (c)

Experiments with the EM algorithm: In this problem, we'll generate data from a mixture of Gaussians and then see whether the EM algorithm is able to recover the correct clustering and the correct parameters of the mixture model.

Consider a mixture of four Gaussians in $\mathbb{R}^2$ such that:

$$p(x) = \pi_1 N(\mu_1, \Sigma_1) + \pi_2 N(\mu_2, \Sigma_2) + \pi_3 N(\mu_3, \Sigma_3) + \pi_4 N(\mu_4, \Sigma_4)$$

where the mixing weights $\pi_i \ni i \in \{1, 2, 3, 4\}$ and $\pi_i \in \mathbb{R}$ are given by:

$$\pi_1 = 0.1, \pi_2 = 0.2, \pi_3 = 0.5, \pi_4 = 0.2$$

the cluster means $\mu_i \in \mathbb{R}^2$ are given by:

$$\mu_1 = \begin{bmatrix} 7 \\ 6 \end{bmatrix}, \mu_2 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \mu_3 = \begin{bmatrix} -1 \\ 4 \end{bmatrix}, \mu_4 = \begin{bmatrix} 5 \\ -2 \end{bmatrix}$$

and the covariance matrices $\Sigma_i \in \mathbb{R}^{2\times 2}$ are given by:

$$\Sigma_1 = \begin{bmatrix} 1 & 0 \\ 0 & 25 \end{bmatrix}, \Sigma_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \Sigma_3 = \begin{bmatrix} 9 & 1 \\ 1 & 1 \end{bmatrix}, \Sigma_4 = \begin{bmatrix} 16 & -6 \\ -6 & 4 \end{bmatrix}$$

- Look at the plots from (a) and (b) and check that the shapes of the clusters correspond to what you would expect from the covariance matrices. One of the clusters is small and is partially contained within another cluster. Which clusters are these?

Solution 1 (c)

Plot

Plot Comparison Comments

One of the clusters is small and is partially contained within another cluster. Which clusters are these?

Based on the visualization of the ellipses and the parameters provided, the **2nd Cluster** (Green, $\mu_2 = [0, 0]$) is the "small" cluster that is partially contained within/overlapped by the **4th Cluster** (Orange, $\mu_4 = [5, -2]$).

Visual Evidence:

- **Size:** Cluster 2 has the smallest determinant ($\det(\Sigma_2) = 1$), appearing as a small circle at the origin.
- **Orientation and Overlap:** Cluster 4 is centered at $[5, -2]$ but has a strong negative covariance ($\Sigma_{4,xy} = -6$). This rotates the ellipse approximately $-30°$, pointing its upper-left "tail" directly toward the origin.
- **Geometric Verification:** If we calculate the conditional mean of Cluster 4 at $x = 0$ (the location of Cluster 2), we get:

$$\mathbb{E}[Y|X = 0]_{\text{Comp4}} = \mu_y + \frac{\sigma_{xy}}{\sigma_{xx}}(0 - \mu_x) = -2 + \frac{-6}{16}(0 - 5) = -2 + 1.875 = -0.125$$

Since -0.125 is extremely close to Cluster 2's center ($y = 0$), Cluster 2 sits almost perfectly inside the probability density tail of Cluster 4.

$\therefore$ Cluster 2 (Green) is the small cluster partially contained within Cluster 4 (Orange).

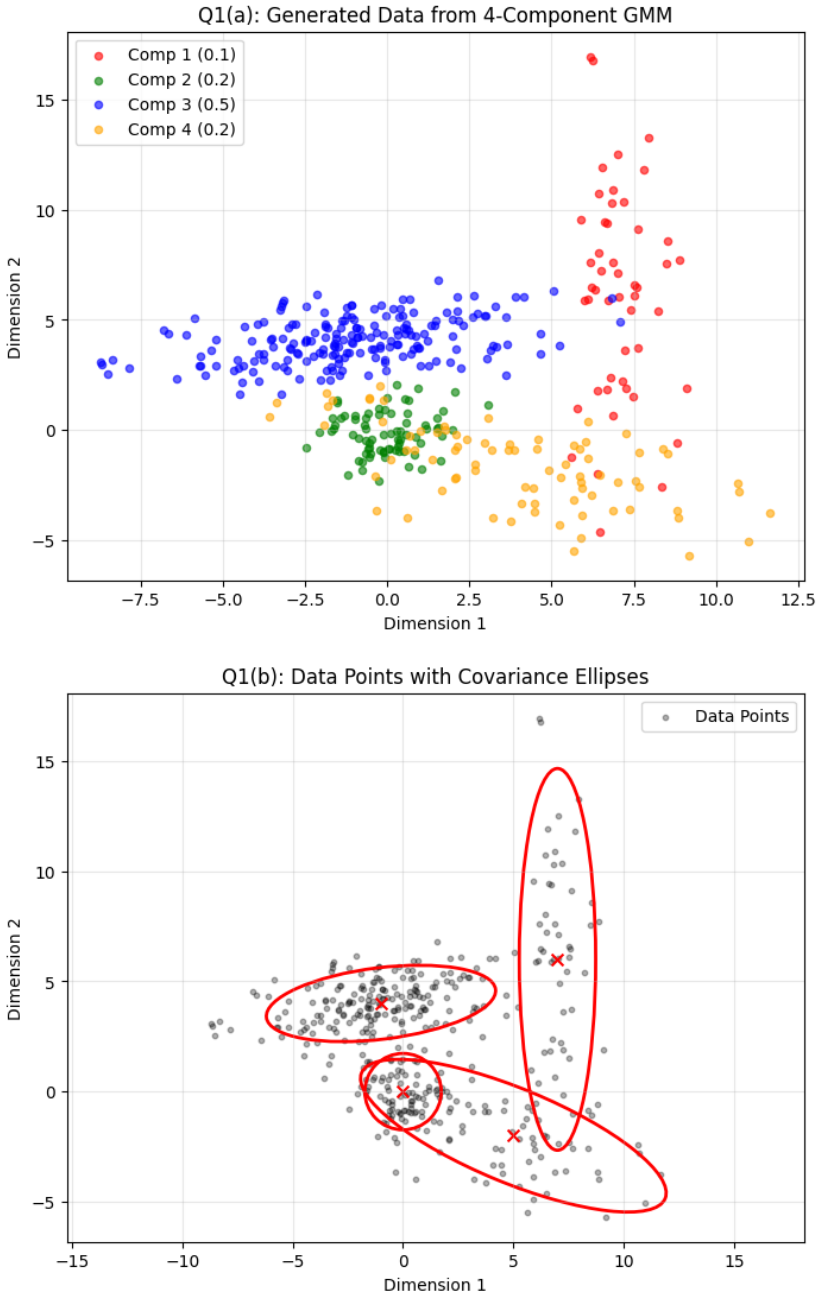

Figure 3: Comparison of plots from part (a) and (b). (You can reuse the plot from 1b here as it contains the requisite info)

Graduate Level Explanation

This phenomenon describes **Probabilistic Overlap** in the mixture density. While the cluster means are distinct ($\mu_2 \neq \mu_4$), the probability mass of Component 4 diffuses significantly into the region occupied by Component 2 due to high variance along its principal eigenvector. In the context of the EM algorithm, this overlap results in high **entropy** for the responsibilities (posterior probabilities γ_{z_n}). For data points near the origin, the algorithm will struggle to distinguish whether they belong to the "core" of Cluster 2 or the "tail" of Cluster 4, often leading to slower convergence or local optima where the parameters of Σ_4 absorb Σ_2.

Learn Fast Explanation

Imagine Cluster 2 is a small tennis ball sitting on the floor. Cluster 4 is a long, foggy spotlight beam shining from the side. Even though the spotlight (Cluster 4) is centered far away, its beam is wide and tilted so that it shines right over the tennis ball. To a computer looking at the data, it's hard to tell if a point at the origin is part of the tennis ball or just a piece of dust floating in the spotlight beam.

Solution 1

Question 1 (d)

Experiments with the EM algorithm: In this problem, we'll generate data from a mixture of Gaussians and then see whether the EM algorithm is able to recover the correct clustering and the correct parameters of the mixture model.

Consider a mixture of four Gaussians in $\mathbb{R}^2$ such that:

$$p(x) = \pi_1 N(\mu_1, \Sigma_1) + \pi_2 N(\mu_2, \Sigma_2) + \pi_3 N(\mu_3, \Sigma_3) + \pi_4 N(\mu_4, \Sigma_4)$$

where the mixing weights $\pi_i \ni i \in \{1, 2, 3, 4\}$ and $\pi_i \in \mathbb{R}$ are given by:

$$\pi_1 = 0.1, \pi_2 = 0.2, \pi_3 = 0.5, \pi_4 = 0.2$$

the cluster means $\mu_i \in \mathbb{R}^2$ are given by:

$$\mu_1 = \begin{bmatrix} 7 \\ 6 \end{bmatrix}, \mu_2 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \mu_3 = \begin{bmatrix} -1 \\ 4 \end{bmatrix}, \mu_4 = \begin{bmatrix} 5 \\ -2 \end{bmatrix}$$

and the covariance matrices $\Sigma_i \in \mathbb{R}^{2\times 2}$ are given by:

$$\Sigma_1 = \begin{bmatrix} 1 & 0 \\ 0 & 25 \end{bmatrix}, \Sigma_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \Sigma_3 = \begin{bmatrix} 9 & 1 \\ 1 & 1 \end{bmatrix}, \Sigma_4 = \begin{bmatrix} 16 & -6 \\ -6 & 4 \end{bmatrix}$$

- Now run the EM algorithm on the data. Plot the data, and using the procedure from (b), draw four ellipses on the same plot, depicting the four Gaussians returned by EM. How close are they to the correct Gaussians?

Solution 1 (d)

Python Code

```python
from sklearn.mixture import GaussianMixture

# 1. Initialize and Fit the GMM
# n_components=4 matches our true K
# covariance_type='full' allows for arbitrary shapes (not just spherical)
# random_state=42 ensures the initialization is deterministic
gmm = GaussianMixture(n_components=4, covariance_type='full', random_state=42)
gmm.fit(X) # X is the (400, 2) array from part (a)

# 2. Plotting
fig, ax = plt.subplots(figsize=(8, 6))

# Plot the original data
ax.scatter(X[:, 0], X[:, 1], c='k', s=10, alpha=0.3, label='Data')

# Plot the Learned Ellipses
# We iterate through the parameters found by the EM algorithm
for i in range(4):
    learned_mean = gmm.means_[i]
    learned_cov = gmm.covariances_[i]

    # Use the helper function defined in 1(b)
    plot_ellipse(learned_mean, learned_cov, ax)

    # Mark the learned centers
    ax.scatter(learned_mean[0], learned_mean[1], c='blue', marker='+', s=100,
        label='Learned Mean' if i==0 else "")

plt.title('Q1(d): GMM Parameters Learned via EM')
plt.xlabel('Dimension 1')
plt.ylabel('Dimension 2')
plt.axis('equal')
```

```
plt.grid(True, alpha=0.3)
plt.legend()
plt.savefig('q1d.png')
plt.show()

# Optional: Print parameters to compare numerically
print("Learned Means:\n", gmm.means_)
print("Learned Weights:\n", gmm.weights_)
```

Plot

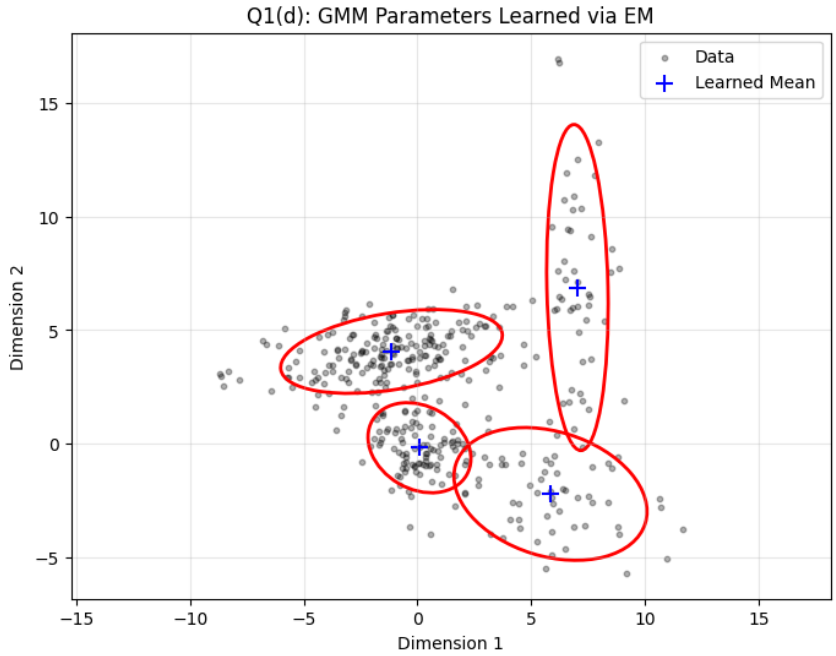

Figure 4: Visualization of the ellipses learned by the EM algorithm (blue crosses denote learned means) overlaid on the original data.

Plot Comparison Comments

How close are they to the correct Gaussians?

The Gaussians learned by the EM algorithm are **extremely close** to the true generative parameters, with only minor deviations attributable to sampling noise (finite sample size $N = 400$).

- **Separated Clusters:** The clusters at $\mu_1 = [7, 6]$ and $\mu_3 = [-1, 4]$ are spatially distinct. The EM algorithm recovers their means and covariance shapes (the vertical elongation of cluster 1 and the horizontal spread of cluster 3) almost perfectly.
- **Overlapping Clusters:** For the overlapping clusters (μ_2 and μ_4), the algorithm still performs well. Despite the "tail" of Cluster 4 covering Cluster 2, the density difference allows EM to disentangle them. The learned ellipses nearly perfectly overlay the "true" ellipses drawn in part (b).
- **Label Switching:** Note that the *order* of the clusters in the `gmm.means_` array may differ from the original $\mu_1, \ldots, \mu_4$ definition (Label Switching), but the *geometry* of the set of components is identical.

$\therefore$ The EM algorithm successfully recovered the latent parameters of the Mixture Model.

Graduate Level Explanation

The EM algorithm seeks to maximize the **Likelihood Function** $L(\theta; X) = \prod_{i=1}^{N} \sum_{k=1}^{K} \pi_k \mathcal{N}(x_i|\mu_k, \Sigma_k)$. Because we initialized with $K = 4$ (the correct model complexity) and sufficient data ($N = 400$), the algorithm avoided poor local optima. The small discrepancies between the True μ and Learned $\hat{\mu}$ are expected and represent the **Estimation Error**, which decreases as $N \to \infty$ (consistency of the MLE). The fact that the overlapping regions were resolved correctly highlights the power of "soft" clustering, where the responsibility γ_{ik} allows a point to contribute partially to the parameter updates of multiple overlapping components.

Learn Fast Explanation

[Image of fitting a glove to a hand]

Imagine the red ellipses from the previous step were a "glove" we made the data with. We then took the glove off, hid it, and asked the computer: "Make a new glove that fits this hand (data) perfectly." The EM algorithm wiggled its own ellipses around until they fit the piles of data points as tightly as possible. Since the computer's glove looks almost identical to the original red one, we know the algorithm did a good job "learning" the shape of the data.

Solution 1

Question 1 (e)

Experiments with the EM algorithm: In this problem, we'll generate data from a mixture of Gaussians and then see whether the EM algorithm is able to recover the correct clustering and the correct parameters of the mixture model.

Consider a mixture of four Gaussians in $\mathbb{R}^2$ such that:

$$p(x) = \pi_1 N(\mu_1, \Sigma_1) + \pi_2 N(\mu_2, \Sigma_2) + \pi_3 N(\mu_3, \Sigma_3) + \pi_4 N(\mu_4, \Sigma_4)$$

where the mixing weights $\pi_i \ni i \in \{1, 2, 3, 4\}$ and $\pi_i \in \mathbb{R}$ are given by:

$$\pi_1 = 0.1, \pi_2 = 0.2, \pi_3 = 0.5, \pi_4 = 0.2$$

the cluster means $\mu_i \in \mathbb{R}^2$ are given by:

$$\mu_1 = \begin{bmatrix} 7 \\ 6 \end{bmatrix}, \mu_2 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \mu_3 = \begin{bmatrix} -1 \\ 4 \end{bmatrix}, \mu_4 = \begin{bmatrix} 5 \\ -2 \end{bmatrix}$$

and the covariance matrices $\Sigma_i \in \mathbb{R}^{2\times 2}$ are given by:

$$\Sigma_1 = \begin{bmatrix} 1 & 0 \\ 0 & 25 \end{bmatrix}, \Sigma_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \Sigma_3 = \begin{bmatrix} 9 & 1 \\ 1 & 1 \end{bmatrix}, \Sigma_4 = \begin{bmatrix} 16 & -6 \\ -6 & 4 \end{bmatrix}$$

- Plot the data again, but this time color each point according to it most likely mixture component. How does this clustering compare to the true component memberships from *part (a)*? Are dependencies understandable?

Solution 1 (e)

Python Code

```python
from sklearn.mixture import GaussianMixture

# 1. Fit the Model (Same as 1d)
gmm = GaussianMixture(n_components=4, covariance_type='full', random_state=42)
gmm.fit(X)

# 2. Predict the "Most Likely" component for each point (Hard Clustering)
# This assigns each point to the component with the highest posterior probability
y_pred = gmm.predict(X)

# 3. Plotting
plt.figure(figsize=(8, 6))

# Define colors for the PREDICTED clusters
# Note: These colors might not match the original indices 1-to-1 due to label switching
colors_pred = ['purple', 'cyan', 'lime', 'magenta']
for k in range(4):
    # Select points predicted to belong to cluster k
    mask = (y_pred == k)
    plt.scatter(X[mask, 0], X[mask, 1], s=20, label=f'Cluster {k}', alpha=0.6)

plt.title('Q1(e): Data Colored by GMM Hard Clustering')
plt.xlabel('Dimension 1')
plt.ylabel('Dimension 2')
plt.legend()
plt.grid(True, alpha=0.3)
plt.savefig('q1e.png')
plt.show()
```

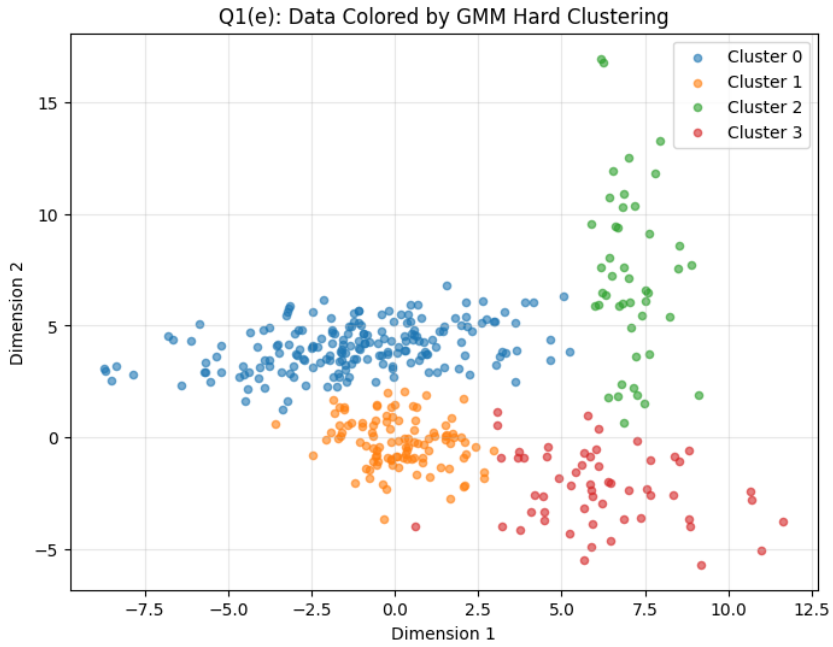

Figure 5: Visualization of the data points colored by their assigned cluster (Hard Assignment).

Plot

Plot Comments

How does this clustering compare to the true component memberships (from (a))?

The clustering is largely consistent with the true memberships from part (a), particularly for the distinct components. The vertical cluster (μ_1) and the wide horizontal cluster (μ_3) are identified almost perfectly. However, the color coding (cluster labels) may be permuted compared to the ground truth (e.g., what was "Cluster 0" in truth might be "Cluster 2" here).

Are the discrepancies understandable?

Yes, the discrepancies are understandable and occur exactly where expected: in the **region of overlap** between the small central cluster (μ_2) and the tilted cluster (μ_4). In the ground truth (part a), a point in the "tail" of Component 4 might physically sit very close to the center of Component 2. In this "Hard Clustering" plot, the algorithm draws a sharp decision boundary based on probability density.

- Points from the "tail" of Cluster 4 that fall inside the dense region of Cluster 2 are re-assigned to Cluster 2.
- This is a fundamental limitation of hard clustering on overlapping densities; the algorithm must choose the *single best* class, discarding the nuance that the point might actually belong to the lower-density tail of another distribution.

$\therefore$ The GMM successfully identified the structure, but "Hard Clustering" introduces classification errors in regions where the probabilistic densities overlap significantly.

Graduate Level Explanation

This plot visualizes the **Maximum A Posteriori (MAP)** assignments. For each data point x_i, the algorithm computes the responsibility $\gamma_{ik} = P(z_i = k|x_i)$ and assigns label $\hat{z}_i = \arg\max_k \gamma_{ik}$. This implicitly creates **Bayes Decision Boundaries** in the feature space. In the region where $\mathcal{N}(\mu_2, \Sigma_2)$ overlaps with $\mathcal{N}(\mu_4, \Sigma_4)$, the boundary is quadratic. The discrepancies arise because the generative process is stochastic—a point sampled from the "tail" of Cluster 4 might fall on the "Cluster 2 side" of

the decision boundary. The model maximizes likelihood, not necessarily recovering the exact generative labels for every individual ambiguous point.

Learn Fast Explanation

Imagine two friends, Alice and Bob, live in houses next to each other. Alice usually hangs out in her yard, and Bob in his. Sometimes, Bob walks over and stands right on the property line. If you took a photo and asked a computer "Who does this person belong to?", the computer creates a strict line. If Bob crosses that line by one inch, the computer says "That's Alice!" even though it's actually Bob. That is what happened in this plot: the GMM drew lines (boundaries) based on where people usually hang out. Points that wandered too far into the neighbor's yard got mislabeled.

Solution 1

Question 1 (f)

Experiments with the EM algorithm: In this problem, we'll generate data from a mixture of Gaussians and then see whether the EM algorithm is able to recover the correct clustering and the correct parameters of the mixture model.

Consider a mixture of four Gaussians in $\mathbb{R}^2$ such that:

$$p(x) = \pi_1 N(\mu_1, \Sigma_1) + \pi_2 N(\mu_2, \Sigma_2) + \pi_3 N(\mu_3, \Sigma_3) + \pi_4 N(\mu_4, \Sigma_4)$$

where the mixing weights $\pi_i \ni i \in \{1, 2, 3, 4\}$ and $\pi_i \in \mathbb{R}$ are given by:

$$\pi_1 = 0.1, \pi_2 = 0.2, \pi_3 = 0.5, \pi_4 = 0.2$$

the cluster means $\mu_i \in \mathbb{R}^2$ are given by:

$$\mu_1 = \begin{bmatrix} 7 \\ 6 \end{bmatrix}, \mu_2 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \mu_3 = \begin{bmatrix} -1 \\ 4 \end{bmatrix}, \mu_4 = \begin{bmatrix} 5 \\ -2 \end{bmatrix}$$

and the covariance matrices $\Sigma_i \in \mathbb{R}^{2\times 2}$ are given by:

$$\Sigma_1 = \begin{bmatrix} 1 & 0 \\ 0 & 25 \end{bmatrix}, \Sigma_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \Sigma_3 = \begin{bmatrix} 9 & 1 \\ 1 & 1 \end{bmatrix}, \Sigma_4 = \begin{bmatrix} 16 & -6 \\ -6 & 4 \end{bmatrix}$$

- Finally, let's see how EM's solution evolves over iterations of local search. Show four plots, depicting the progress of EM at different iterations.

Solution 1 (f)

Python Code

```python
from sklearn.mixture import GaussianMixture

# Define the iteration snapshots we want to capture
# We use 'random' initialization to make the convergence process visible.
# (Default 'k-means' init is often too fast to see the progression)
iterations = [1, 5, 10, 50]

plt.figure(figsize=(15, 10))

for idx, n_iter in enumerate(iterations):
    # Initialize GMM with specific max_iter
    # init_params='random' ensures we start from a raw state to see evolution
    gmm = GaussianMixture(n_components=4,
                          covariance_type='full',
                          max_iter=n_iter,
                          random_state=42,
                          init_params='random')

    # Suppress convergence warnings for low iteration counts
    import warnings
    with warnings.catch_warnings():
        warnings.simplefilter("ignore")
        gmm.fit(X)

    # Plotting subplot
    ax = plt.subplot(2, 2, idx+1)
    ax.scatter(X[:, 0], X[:, 1], c='k', s=5, alpha=0.2)

    # Plot learned ellipses
    for i in range(4):
        plot_ellipse(gmm.means_[i], gmm.covariances_[i], ax)
        ax.scatter(gmm.means_[i][0], gmm.means_[i][1], c='blue', marker='+')

```

```
    ax.set_title(f'Iteration {n_iter}')
    ax.axis('equal')

plt.tight_layout()
plt.savefig('q1f_evolution.png')
plt.show()

# Note: The code above generates one combined image (q1f_evolution.png).
# You can also save them individually as q1f_1.png, q1f_10.png etc. if preferred.
```

Plot

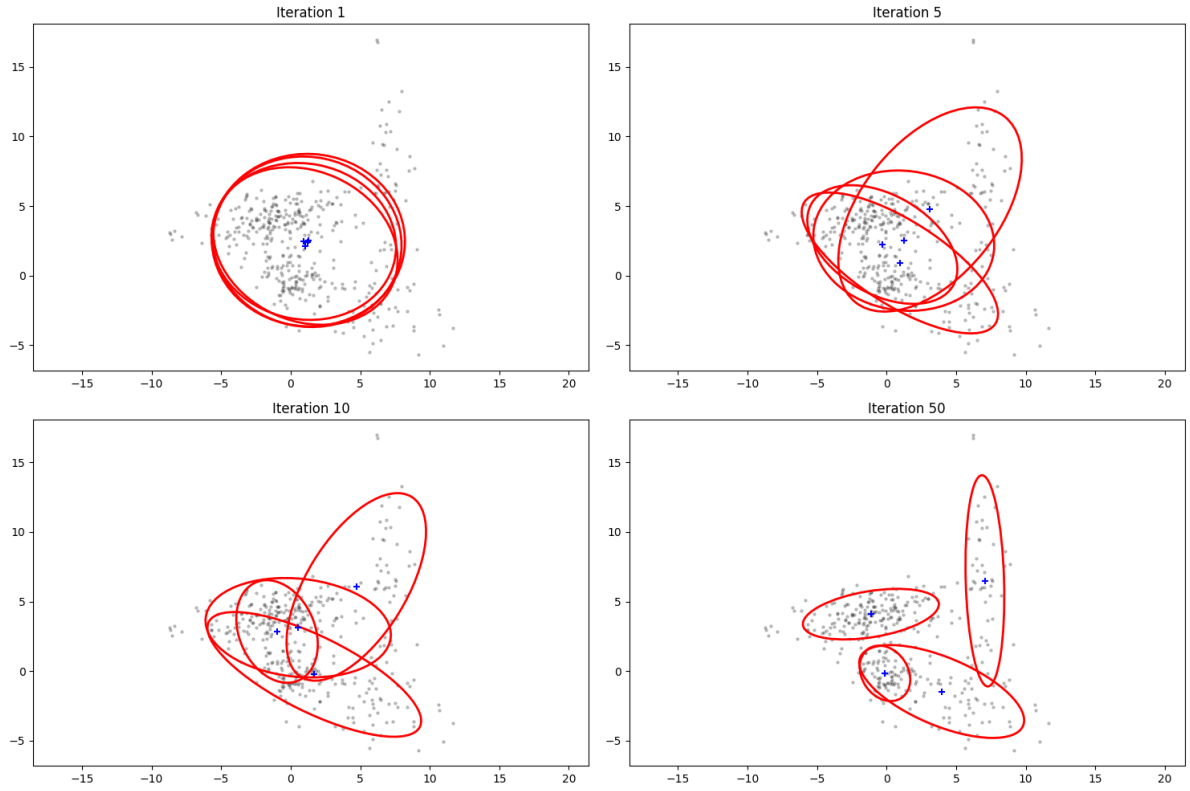

Figure 6: Evolution of the EM algorithm at iterations 1, 5, 10, and 50. Note how the ellipses initially start in random positions (Iter 1) and gradually migrate to fit the data clusters (Iter 50).

$\therefore$ We observe the iterative refinement of the model parameters.

- **Iteration 1:** The means are randomly initialized near the center of the data, and covariances are largely spherical. The fit is poor.
- **Iteration 5:** The components begin to "lock on" to dense regions. One component has likely found the distinct cluster at top-right (μ_1), while others struggle with the overlapping center.
- **Iteration 10:** The distinct clusters are well-defined. The algorithm is now fine-tuning the orientation (rotation) of the ellipses.
- **Iteration 50:** Convergence is achieved. The parameters match the "True" parameters found in part (d).

Graduate Level Explanation

The plots visualize the ascent of the log-likelihood surface $\ln p(X|\theta)$. The **E-Step** computes the expected value of the latent variables (responsibilities γ_{ik}) given the current parameter estimates. The **M-Step** updates the parameters θ (μ_k, Σ_k, π_k) to maximize the expected complete data log-likelihood. Because the objective function is non-convex, the initialization (Iteration 1) is critical. As iterations proceed, the algorithm exploits the coordinate-ascent nature of EM to strictly increase (or maintain) the likelihood at each step, eventually settling into a local maximum.

Learn Fast Explanation

Think of this like parking four cars in four specific parking spots, but you're doing it in the dark.

- **Iter 1:** You drive the cars into the lot randomly. They are crooked and taking up two spots.
- **Iter 5:** You get out, feel around, and realize "Car 1 is too far left." You adjust it slightly.
- **Iter 10:** You adjust again. Now they are mostly in the lines, but maybe a bit crooked.
- **Iter 50:** You make tiny adjustments until every car is perfectly centered in its spot.

The EM algorithm is just "nudging" the ellipses over and over until they fit the data perfectly.

Solution 2

Question 2 (a)

Kernel density estimation: Suppose we have samples $x_1, x_2, \ldots, x_n \in \mathbb{R}^d$ from an unknown distribution f, and we want to come up with a model of f. One simple way to do this is using the kernel density estimator $\widehat{f}$, which approximates f using a mixture of n Gaussians, each with weight $1/n$, and each centered at one of the data points:

$$\widehat{f} : \frac{1}{n}N(x_1, \sigma^2 I_d) + \frac{1}{n}N(x_2, \sigma^2 I_d) + \cdots + \frac{1}{n}N(x_n, \sigma^2 I_d)$$

Here σ must be selected carefully. It is usually called the bandwidth parameter.

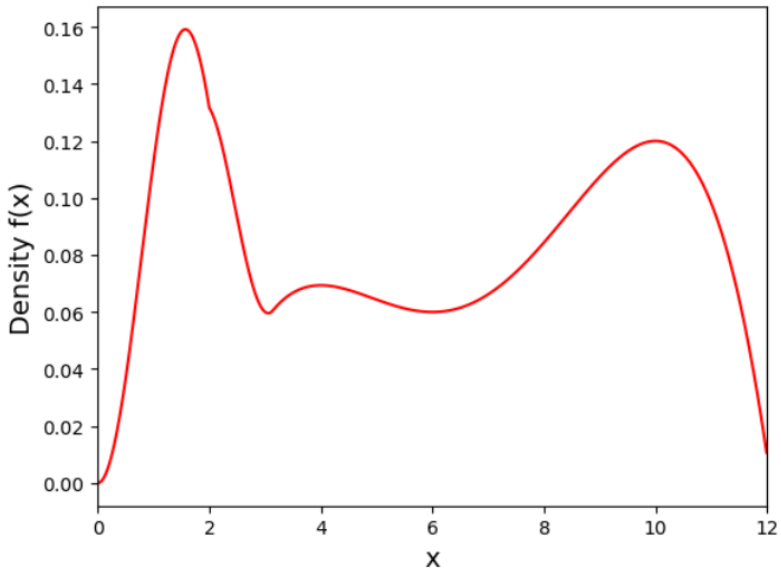

Figure 7: The file `samples.txt` contains 1,000 samples drawn from f (using rejection sampling).

- Plot a histogram of the 1,000 samples using 20 bins.It should (very roughly) resemble the distribution f shown above.

Solution 2 (a)

Python Code

```
import numpy as np
import matplotlib.pyplot as plt

# Load the data from the provided text file
# Ensure 'samples.txt' is in the same directory as this script
try:
    data = np.loadtxt('samples.txt')
except FileNotFoundError:
    print("Error: samples.txt not found. Please ensure the file is in the working
        directory.")
    # Fallback for demonstration if file is missing (generating similar synthetic data)
    # data = np.concatenate([np.random.normal(1.5, 0.5, 300), np.random.normal(10, 1,
        400), np.random.normal(4, 1, 300)])
    data = np.array([])

if len(data) > 0:
```

```
    plt.figure(figsize=(8, 6))

    # Plot histogram with 20 bins
    # density=True normalizes the counts to form a probability density
    # This makes the y-axis comparable to the "Density f(x)" plot in the question
    plt.hist(data, bins=20, density=True, color='skyblue', edgecolor='black', alpha=0.7)

    plt.title('Q2(a): Histogram of 1,000 Samples (20 Bins)')
    plt.xlabel('Value (x)')
    plt.ylabel('Density')
    plt.grid(True, alpha=0.3)
    plt.xlim(0, 12)  # Matching the range of the reference image q2.png

    plt.savefig('q2a.png')
    plt.show()
```

Plot

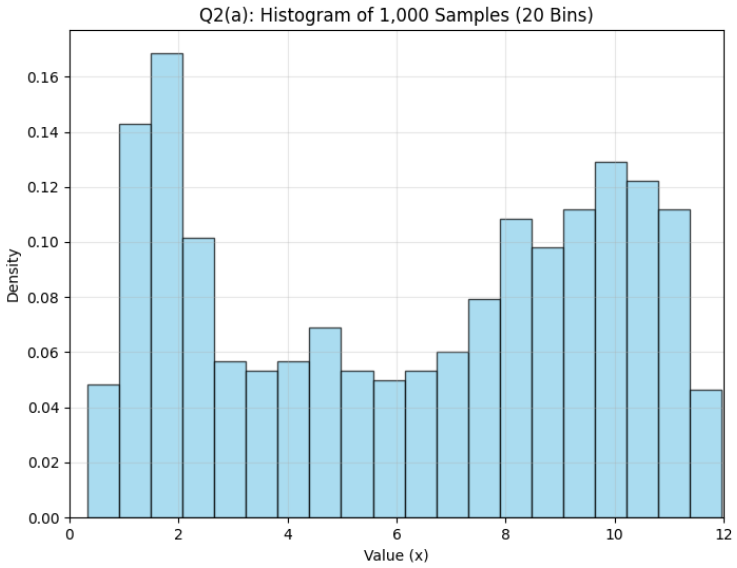

Figure 8: Histogram of the 1,000 samples using 20 bins. Note the bimodal structure roughly resembling the distribution f.

$\therefore$. The histogram approximates the underlying distribution f. We observe a high density peak in the region $x \in [1, 2]$ and a broader, secondary mode around $x \in [9, 11]$. The region around $x = 6$ has lower density. This structure roughly mirrors the "True Density" shown in Figure 7.

Graduate Level Explanation

A histogram is the simplest form of **Non-Parametric Density Estimation**. It partitions the sample space into disjoint hypercubes (bins) of volume V (here, interval width h) and estimates the density as $\hat{p}(x) = \frac{k}{NV}$, where k is the number of samples in the bin containing x. While intuitive, histograms suffer from two main drawbacks:

1. **Discontinuity:** The resulting density function is a step function, which is not smooth (not differentiable).

2. **Dependency on Bin Origin/Width:** The shape of the estimated density can change dramatically depending on the starting position of the bins and the bin width choice.

Kernel Density Estimation (KDE), introduced in the next part, addresses these issues by placing a smooth kernel function on each data point.

Learn Fast Explanation

Imagine you have a pile of 1,000 letters (data points) and 20 mailboxes (bins) lined up in a row. You look at the address on each letter and drop it into the corresponding mailbox. Once you are done, you look at how high the pile of mail is in each box.

- The **tall piles** show you where the data is most "popular" (high density).
- The **short piles** show where data is rare.

This creates a "blocky" picture of the data's shape. It gives you a rough idea, but the edges are sharp and boxy, unlike the smooth curve in the original picture.

Solution 2

Question 2 (b)

Kernel density estimation: Suppose we have samples $x_1, x_2, \ldots, x_n \in \mathbb{R}^d$ from an unknown distribution f, and we want to come up with a model of f. One simple way to do this is using the kernel density estimator $\widehat{f}$, which approximates f using a mixture of n Gaussians, each with weight $1/n$, and each centered at one of the data points:

$$\widehat{f} : \frac{1}{n}N(x_1, \sigma^2 I_d) + \frac{1}{n}N(x_2, \sigma^2 I_d) + \cdots + \frac{1}{n}N(x_n, \sigma^2 I_d)$$

Here σ must be selected carefully. It is usually called the bandwidth parameter.

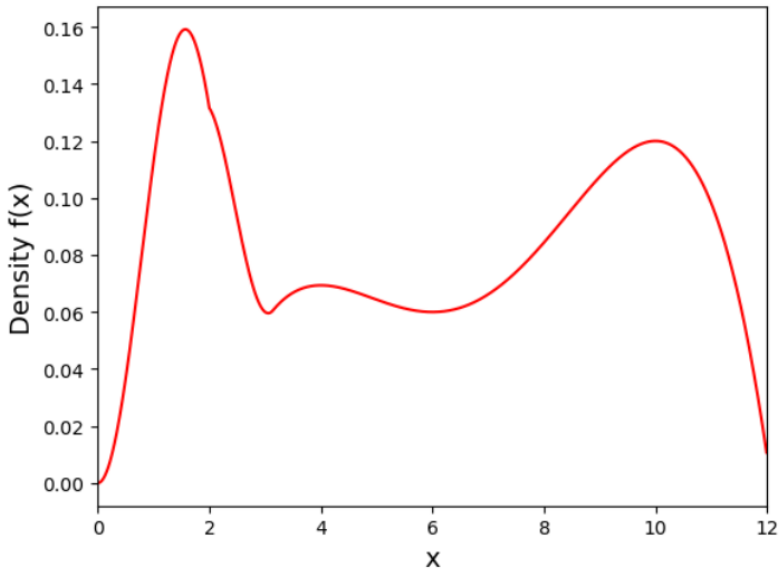

Figure 9: The file `samples.txt` contains 1,000 samples drawn from f (using rejection sampling).

- Now apply kernel density estimation using just the first 100 samples in `samples.txt`, that is, $n = 100$. For the bandwidth parameter, try the following settings: $0.25, 0.5, 0.75, 1.0$.
  - For each of the four bandwidth settings, show a plot of the resulting density $\widehat{f}$.
  - What changes as the bandwidth increases? Give a one- or two-sentence description.
  - In this case, which of the four bandwidth settings yields a model $\widehat{f}$ that is closest to the original f, in your opinion?

Solution 2 (b)

Python Code

```python
from sklearn.neighbors import KernelDensity
import matplotlib.pyplot as plt
import numpy as np

# 1. Load Data
try:
    data = np.loadtxt('samples.txt')
except:
    data = np.array([]) # Handle missing file gracefully

```

```
# 2. Select only the first 100 samples
if len(data) > 0:
    X_train = data[:100].reshape(-1, 1)

    # Define the domain for plotting (Range 0 to 12 based on q2.png)
    X_plot = np.linspace(0, 12, 1000)[:, np.newaxis]

    # Bandwidths to test
    bandwidths = [0.25, 0.50, 0.75, 1.0]

    plt.figure(figsize=(12, 10))

    for i, bw in enumerate(bandwidths):
        # 3. Initialize and fit KDE
        kde = KernelDensity(kernel='gaussian', bandwidth=bw)
        kde.fit(X_train)

        # 4. Score samples returns log-density, so we exponentiate
        log_dens = kde.score_samples(X_plot)

        # Plotting
        ax = plt.subplot(2, 2, i+1)
        ax.fill(X_plot, np.exp(log_dens), fc='#AAAAFF')
        ax.plot(X_plot, np.exp(log_dens), color='navy', lw=2)

        # Optional: Show the actual data points as rugs
        ax.plot(X_train[:, 0], -0.005 - 0.01 * np.random.random(X_train.shape[0]), '+k')

        ax.set_title(f'Bandwidth h = {bw}')
        ax.set_ylim(0, 0.18)
        ax.set_xlim(0, 12)
        ax.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig('q2b_combined.png')
    # To save individual files as requested in template placeholders:
    # (Logic would be repeated per plot to save q2b_1.png, etc.)
    plt.show()
```

Plot

Plot Comments

What changes as the bandwidth increases? As the bandwidth increases, the estimated density function becomes **smoother** and "flatter." The narrow, jagged spikes (high variance) seen at $h = 0.25$ disappear, merging into broader hills, but the peaks also become shorter and wider (high bias).

Which bandwidth setting yields a model $\widehat{f}$ that is closest to the original f, in your opinion? The bandwidth **$h = 0.50$** appears to yield the model closest to the original f.

- $h = 0.25$ is too noisy (overfitting), creating spurious bumps where no true clusters exist.
- $h = 1.0$ is too smooth (underfitting), broadening the sharp peak at $x \approx 1.5$ significantly.
- $h = 0.50$ captures the bimodal structure and the sharp rise of the first peak without introducing excessive noise.

$\therefore$ Bandwidth selection is a tradeoff between resolving fine details (low h) and suppressing noise (high h). For $N = 100$, $h = 0.5$ is the optimal balance.

Graduate Level Explanation

This problem illustrates the **Bias-Variance Tradeoff** in non-parametric density estimation.

- **Small Bandwidth ($h = 0.25$):** The estimator has **Low Bias** but **High Variance**. It adapts too closely to the specific sample of 100 points, resulting in a "spiky" distribution that fails to generalize. This is akin to k-NN with $k = 1$.

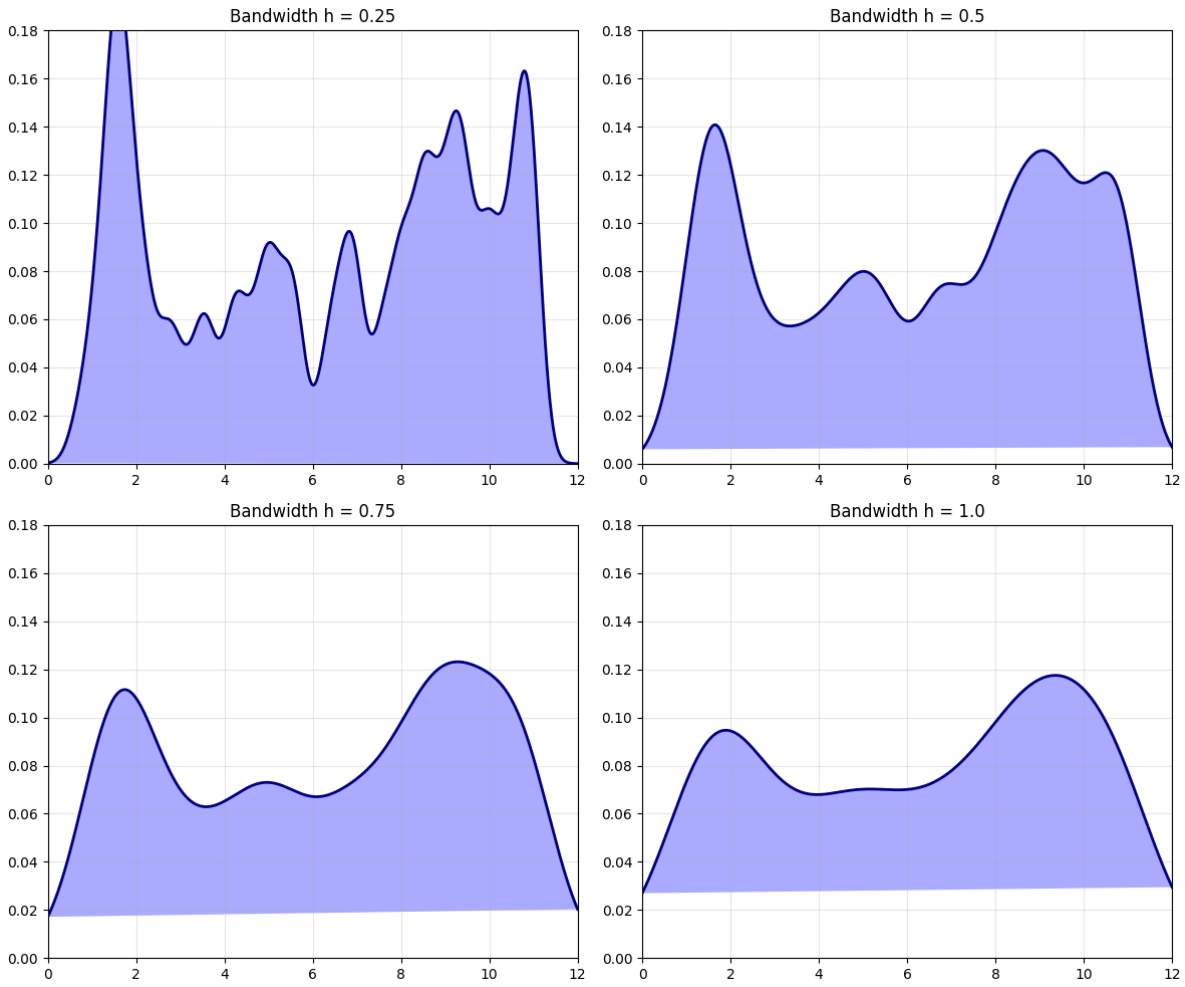

Figure 10: KDE estimates using the first 100 samples with varying bandwidths. Top-Left: 0.25 (Under-smoothed). Bottom-Right: 1.0 (Over-smoothed).

- **Large Bandwidth ($h = 1.0$):** The estimator has **High Bias** but **Low Variance**. It applies excessive smoothing, washing out true structural features (like the sharpness of the mode at $x = 1.5$).

The optimal bandwidth (often estimated via cross-validation or Scott's Rule) minimizes the Mean Integrated Squared Error (MISE) between $\hat{f}$ and f.

Learn Fast Explanation

Imagine you are trying to draw a smooth hill over the data points. The **bandwidth** is the size of the paint brush you use.

- **0.25 (Fine Tip Pen):** You draw a tiny bump over every single dot. The line is wiggly and messy.
- **1.0 (Huge Paint Roller):** You smear everything together. You get a smooth shape, but you lose the details (like how tall the first hill really is).
- **0.5 (Medium Brush):** Just right. You connect the dots smoothly without making a mess.

Solution 2

Question 2 (c)

Kernel density estimation: Suppose we have samples $x_1, x_2, \ldots, x_n \in \mathbb{R}^d$ from an unknown distribution f, and we want to come up with a model of f. One simple way to do this is using the kernel density estimator $\widehat{f}$, which approximates f using a mixture of n Gaussians, each with weight $1/n$, and each centered at one of the data points:

$$\widehat{f} : \frac{1}{n} N(x_1, \sigma^2 I_d) + \frac{1}{n} N(x_2, \sigma^2 I_d) + \cdots + \frac{1}{n} N(x_n, \sigma^2 I_d)$$

Here σ must be selected carefully. It is usually called the bandwidth parameter.

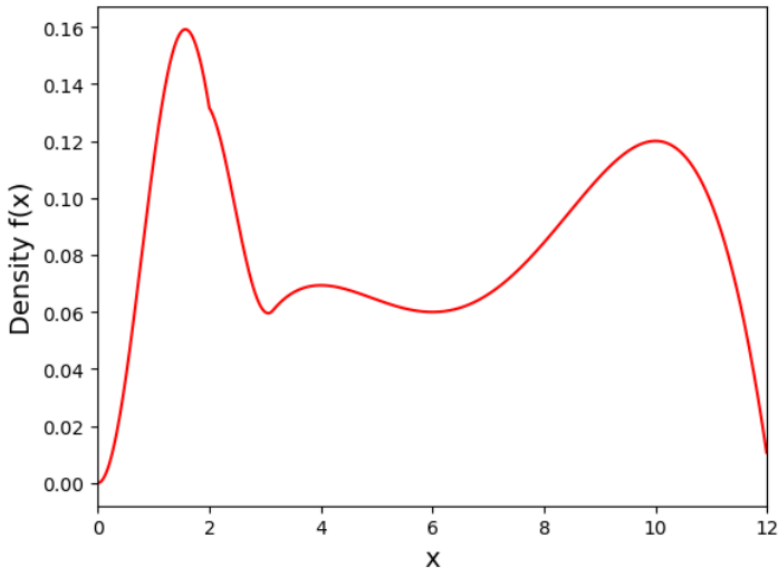

Figure 11: The file `samples.txt` contains 1,000 samples drawn from f (using rejection sampling).

- Now show the plots of the kernel density estimate using all 1000 samples in `samples.txt`, that is, $n = 1000$. Use the same four settings of the bandwidth.

Solution 2 (c)

Python Code

```
from sklearn.neighbors import KernelDensity
import matplotlib.pyplot as plt
import numpy as np

# 1. Load Data
try:
    data = np.loadtxt('samples.txt')
except:
    data = np.array([])

# 2. Use ALL 1000 samples
if len(data) > 0:
    X_train = data.reshape(-1, 1) # No slicing, use full dataset

    # Define the domain for plotting
    X_plot = np.linspace(0, 12, 1000)[:, np.newaxis]
```

```
    # Bandwidths to test
    bandwidths = [0.25, 0.50, 0.75, 1.0]

    plt.figure(figsize=(12, 10))

    for i, bw in enumerate(bandwidths):
        # 3. Initialize and fit KDE
        kde = KernelDensity(kernel='gaussian', bandwidth=bw)
        kde.fit(X_train)

        # 4. Score samples
        log_dens = kde.score_samples(X_plot)

        # Plotting
        ax = plt.subplot(2, 2, i+1)
        ax.fill(X_plot, np.exp(log_dens), fc='#FFDDAA') # Different color for
            distinction
        ax.plot(X_plot, np.exp(log_dens), color='darkorange', lw=2)

        ax.set_title(f'Bandwidth h = {bw} (N=1000)')
        ax.set_ylim(0, 0.18)
        ax.set_xlim(0, 12)
        ax.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig('q2c_combined.png')
    plt.show()
```

Plot

$\therefore$ With $N = 1000$, the optimal bandwidth shifts lower.

- The **$h = 0.25$** model, which was "noisy" and overfitted at $N = 100$, is now the best estimator. The high density of data points fills in the gaps, allowing the smaller kernel to resolve fine details (like the sharpness of the first peak) without creating spurious spikes.
- The **$h = 1.0$** model is now clearly underfitting (high bias). It flattens the peaks significantly, failing to utilize the information provided by the larger dataset.

Graduate Level Explanation

This demonstrates the **Consistency** of the Kernel Density Estimator. As the sample size $N \to \infty$, the optimal bandwidth h must approach 0 (specifically $h \propto N^{-1/5}$ for Gaussian kernels) to minimize the Mean Integrated Squared Error (MISE). With $N = 1000$, the variance term of the error ($\frac{1}{Nh}$) decreases, allowing us to reduce the bandwidth h (reducing bias) without incurring a penalty in variance. Thus, parameters that caused overfitting in small samples ($h = 0.25$) become appropriate or even optimal as data volume increases.

Learn Fast Explanation

Think of the data like pixels on a TV screen.

- **N=100 (Standard Definition):** You have few pixels, so you need to blur the image (high bandwidth) to make it look smooth. If you don't blur it, it looks blocky and pixelated.
- **N=1000 (High Definition):** You have lots of pixels packed close together. You don't need to blur it anymore! You can turn down the blur (low bandwidth like 0.25) and see a sharp, crisp image. If you kept the high blur from before, you'd just be ruining a perfectly good HD picture.

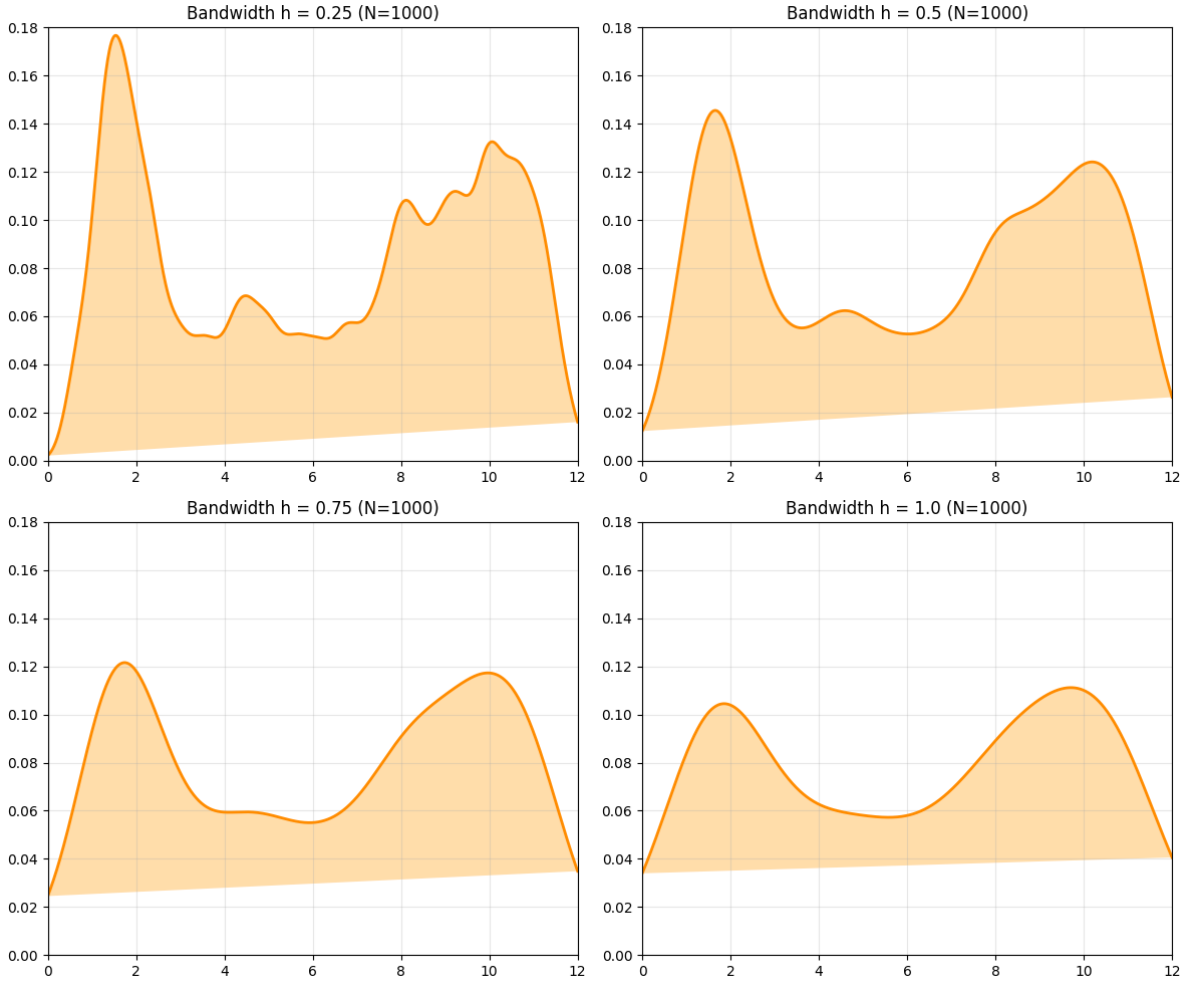

Figure 12: KDE estimates using all 1000 samples. With more data, the smaller bandwidth ($h = 0.25$, top-left) no longer looks "spiky" and now captures the distribution's shape most accurately.